

Redes neuronales aplicadas a la verificación de firmas manuscritas

TRABAJO DE FIN DE GRADO

Curso académico 2025/2026



UNIVERSIDAD
COMPLUTENSE
MADRID

FACULTAD DE CIENCIAS MATEMÁTICAS

GRADO DE INGENIERÍA MATEMÁTICA

Víctor Marcos Albesa

Tutores: Alexandre Salas Bernárdez, Valeriy Makarov Slizneva

Resumen

La firma manuscrita sigue siendo uno de los métodos biométricos más utilizados para la autenticación de identidad. Sin embargo, su vulnerabilidad ante la falsificación representa un riesgo significativo en ámbitos como la banca o la validación de documentos legales. En este trabajo se aborda el estudio matemático, diseño e implementación de un sistema de verificación de firmas basado en técnicas de aprendizaje profundo.

Se desarrolla el marco teórico que sustenta dos arquitecturas de redes neuronales: el perceptrón multicapa (MLP) y las redes neuronales convolucionales (CNN), incluyendo la formalización matemática de sus componentes, funciones de pérdida, algoritmos de optimización y técnicas de regularización. Para la validación experimental, se construyó un conjunto de datos original de 1512 firmas: 756 auténticas generadas por el autor y 756 falsificadas por 63 voluntarios mediante plantillas estandarizadas.

Los resultados muestran una clara superioridad de la CNN frente al MLP en el conjunto de prueba: mientras que este último alcanza una exactitud máxima del 94,74 % con una tasa de falsa aceptación próxima al 10 %, la CNN logra una exactitud del 98,68 % y, al ajustar el umbral de decisión al 80 %, consigue erradicar completamente los falsos positivos. Finalmente, la CNN desarrollada se integró en una aplicación móvil para Android, demostrando su viabilidad en un entorno de uso real.

Abstract

Handwritten signatures remain one of the most widely used biometric methods for identity authentication. However, their vulnerability to forgery poses a significant risk in domains such as banking and legal document validation. This work addresses the mathematical study, design and implementation of an automatic signature verification system based on deep learning techniques.

A theoretical framework is developed for two neural network architectures: the Multilayer Perceptron (MLP) and Convolutional Neural Networks (CNN), including the mathematical formalization of their components, loss functions, optimization algorithms and regularization techniques. For experimental validation, an original dataset of 1512 signatures was constructed, consisting of 756 genuine signatures produced by the author and 756 forged signatures collected from 63 volunteers using standardized templates.

The results show a clear advantage of the CNN over the MLP on the test set: while the latter achieves a maximum accuracy of 94,74 % with a false acceptance rate close to 10 %, the CNN trained with SGD without Dropout reaches 98,68 % accuracy and, by raising the decision threshold to 80 %, produces no false positives on the test set. The best-performing model was integrated into an Android mobile application, demonstrating its feasibility in a real-world deployment scenario.

Índice general

1. Introducción	5
1.1. El problema de la clasificación de firmas	5
1.2. Objetivos del trabajo	6
2. Fundamentos Matemáticos del Perceptrón Multicapa (MLP)	7
2.1. Contexto histórico: De la neurona de McCulloch-Pitts al Perceptrón de Rosenblatt	7
2.2. Modelado matemático del perceptrón	7
2.3. Arquitectura de la red MLP	8
2.3.1. Topología <i>feedforward</i> y estructura de capas	8
2.3.2. Formalización matricial del modelo	10
2.4. Funciones de activación	11
2.5. Teorema de Aproximación Universal	12
2.6. Propagación hacia delante (<i>Forward Propagation</i>)	13
2.6.1. Inicialización: capa de entrada	13
2.6.2. Propagación en capas ocultas	14
2.6.3. Capa de salida y predicción final	14
2.7. Funciones de pérdida	15
2.7.1. El Error Cuadrático Medio (MSE) y sus limitaciones	15
2.7.2. Entropía Cruzada Binaria (BCE): Formulación y propiedades	16
3. Fundamentos Matemáticos de las Redes Convolucionales (CNN)	17
3.1. Limitaciones del MLP en el procesamiento de imágenes	17
3.2. La operación de convolución discreta en 2D	17
3.2.1. Matrices de Entrada y <i>Kernel</i>	17
3.2.2. Operación de convolución	18
3.2.3. Hiperparámetros geométricos (<i>Padding</i> y <i>Stride</i>)	18
3.2.4. Mapa de Características (<i>Feature Map</i>) y Pesos Compartidos	19
3.3. Capas de agrupamiento y reducción dimensional	20
3.3.1. Mecánica geométrica y tipos de agrupamientos (<i>Pooling</i>)	20
3.3.2. Reducción dimensional e invarianza espacial	20

3.4.	Arquitectura general de una CNN	21
3.4.1.	Fase de extracción de características	21
3.4.2.	Fase de transición y clasificación	22
4.	Algoritmos de Optimización y Entrenamiento	23
4.1.	El algoritmo de Backpropagation	23
4.1.1.	Objetivo matemático y la Regla de la Cadena	23
4.1.2.	Ciclo de evaluación: <i>Forward</i> y <i>Backward Pass</i>	24
4.2.	La regla Delta y la regla Delta Generalizada	24
4.3.	Descenso de Gradiente Estocástico	25
4.4.	Optimización avanzada: Estimación de Momentos Adaptativos (Adam) . . .	26
4.5.	Regularización matemática: Control del sobreajuste	28
4.5.1.	Penalización L2 (<i>Weight Decay</i>)	28
4.5.2.	Regularización arquitectónica (<i>Dropout</i>)	28
5.	Estudio práctico de firmas manuscritas	29
5.1.	Metodología y generación del <i>Dataset</i>	29
5.2.	Preparación de los datos	30
5.3.	Diseño de las arquitecturas de red	31
5.3.1.	Estructura del Perceptrón Multicapa	31
5.3.2.	Estructura de la Red Neuronal Convolutiva	31
5.4.	Diseño experimental e implementación	32
6.	Resultados prácticos	34
6.1.	Curvas de aprendizaje: Evolución del coste frente al número de épocas . . .	34
6.1.1.	Análisis del Perceptrón Multicapa	34
6.1.2.	Análisis de la Red Neuronal Convolutiva	35
6.2.	Rendimiento global y matrices de confusión	35
6.3.	Análisis de Seguridad Biométrica	36
6.4.	Análisis de eficiencia y coste computacional	37
6.5.	Implementación en Aplicación móvil	37
7.	Conclusiones	39
7.1.	Conclusiones generales del proyecto	39
7.2.	Limitaciones de la implementación en un entorno realista y líneas de trabajo futuras	39

1. Introducción

1.1 El problema de la clasificación de firmas

A pesar del rápido avance hacia la digitalización, la firma manuscrita sigue siendo uno de los métodos biométricos más universales y legalmente aceptados para la autenticación de identidad. Desde la validación de transacciones bancarias y cheques, hasta la firma de contratos hipotecarios o laborales, testamentos o documentos oficiales, nuestra sociedad confía en el trazo de un bolígrafo sobre un papel para confirmar el consentimiento y la autoría de una persona. Sin embargo, esta misma confianza generalizada trae consigo una clara vulnerabilidad: la falsificación. El fraude documental a través de la imitación de firmas manuscritas supone pérdidas millonarias a nivel global y genera complejas disputas legales. El problema radica en que la validación de una firma no es una tarea trivial. Mientras que una falsificación en la que el impostor ni siquiera conoce la firma original es fácil de detectar, una falsificación “entrenada” o elaborada puede suponer un reto mucho mayor. En estos casos, el falsificador ha practicado la dinámica, inclinación y proporciones del trazo original.

Para ilustrar la magnitud de este desafío visual, a continuación se presentan dos muestras (Figura 1.1) del conjunto de datos utilizado (véase la Sección 5.1). A simple vista, ambas pueden parecer trazadas por la misma persona, pero solamente una de ellas es auténtica.

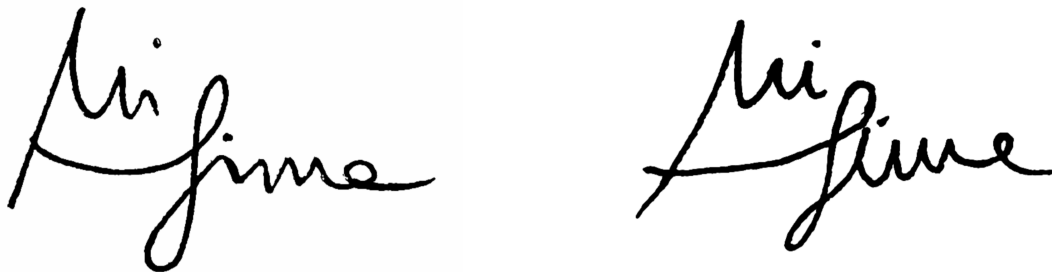


Figura 1.1: Firma auténtica (izquierda) y firma falsificada (derecha)

Tradicionalmente, la verificación de este tipo de documentos recae en peritos calígrafos, especializados en este tipo de problemas. A pesar de que su labor es altamente precisa, al ser un proceso manual, se vuelve lento y costoso, además de que está sujeto a la subjetividad

humana. Por otro lado, este método se torna inviable cuando se necesita verificar miles de transacciones por minuto como sucede, por ejemplo, en el sector bancario. En este escenario, un empleado común o un cajero sin formación específica en grafología tiene una alta tasa de error al realizar esta verificación, por lo que es un trabajo inadecuado para estos perfiles.

Es aquí donde surge la necesidad de la automatización. El problema de la clasificación de firmas se define, por tanto, como el desafío del desarrollo de sistemas capaces de distinguir, de forma rápida, escalable y con precisión sobrehumana, los patrones intrínsecos de firmas auténticas frente a sus falsificaciones. En este contexto exacto, la Inteligencia Artificial, y en particular las Redes Neuronales, ofrecen una solución prometedora.

1.2 Objetivos del trabajo

El objetivo principal de este Trabajo de Fin de Grado es el estudio, diseño e implementación de modelos de aprendizaje profundo, específicamente el Perceptrón Multicapa (MLP) y las Redes Neuronales Convolucionales (CNN), enfocados en la resolución de un problema práctico de clasificación binaria: la verificación de firmas manuscritas para distinguir entre rúbricas auténticas y falsificadas. Para alcanzar esta meta general, el trabajo se desglosa en los siguientes objetivos específicos:

- **Formalización matemática de las Redes Neuronales:** Desarrollo del marco teórico que sustenta los modelos MLP y las arquitecturas convolucionales, definiendo rigurosamente su arquitectura, sus componentes y el papel de cada uno de ellos.
- **Análisis de las funciones de pérdida:** Estudio y contraste matemático de distintas funciones de coste, evaluando sus diferencias y justificando teóricamente su idoneidad en función del problema a resolver.
- **Estudio de la teoría de la optimización:** Desglose de la base matemática de los algoritmos de entrenamiento y métodos de descenso de gradiente, analizando la teoría que los sustenta.
- **Generación y tratamiento del conjunto de datos:** Construcción, curación y pre-procesado de un conjunto de datos (*dataset*) de firmas manuscritas auténticas y falsificadas, abordando las distintas estrategias de adaptación de entrada requeridas por cada modelo.
- **Validación y comparativa arquitectónica:** Aplicación del conocimiento teórico en un entorno computacional real, implementando y contrastando el rendimiento empírico de las redes MLP y CNN para abordar el problema de la verificación de firmas.

2. Fundamentos Matemáticos del Perceptrón Multicapa (MLP)

2.1 Contexto histórico: De la neurona de McCulloch-Pitts al Perceptrón de Rosenblatt

Las redes neuronales artificiales nacen con el modelo propuesto por McCulloch y Pitts en 1943 [1], que representaba la neurona como una unidad binaria operando bajo el principio de “todo o nada”, capaz de computar funciones lógicas, pero carente de un mecanismo de aprendizaje. Rosenblatt amplió esta idea quince años después con el Perceptrón [2], inspirado en la plasticidad sináptica de Hebb [3]. Este modelo introdujo pesos ajustables en función del error, sentando las bases de los clasificadores modernos que se aplican en este trabajo.

2.2 Modelado matemático del perceptrón

El modelo matemático de la neurona artificial, originalmente concebido como perceptrón, es la estructura computacional más elemental dentro de las redes neuronales. Su funcionamiento básico consiste en la proyección de un espacio de entrada multidimensional hacia una decisión binaria [4]. Cada una de las variables de entrada que recibe la neurona contribuye a la decisión final con un cierto nivel de importancia. Para cuantificarla, el modelo asocia un parámetro ajustable denominado peso (*weight*) a cada una de ellas (Figura 2.1).

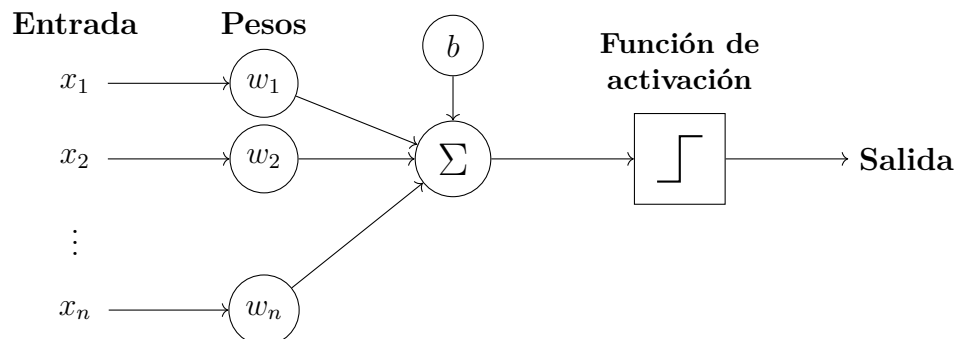


Figura 2.1: Esquema de un perceptrón simple

Formalmente, supongamos que la neurona recibe un vector de entrada $\mathbf{x} = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}^n$, asociado a un vector de pesos $\mathbf{w} = [w_1, w_2, \dots, w_n]^T \in \mathbb{R}^n$. En el contexto del aprendizaje automático, a cada componente x_i se le denomina *característica*. Una característica se define como una propiedad individual, medible e independiente del fenómeno que se está observando. El conjunto de todas ellas conforma el *espacio de características* de dimensión n . En el caso práctico que se estudia en el trabajo, cada característica representa el valor de intensidad de cada píxel de la imagen de la firma (Figura 1.1).

La activación de la neurona viene determinada por si la combinación lineal de las entradas y sus pesos supera un cierto umbral $\theta \in \mathbb{R}$. Es decir:

$$\text{Salida} = \begin{cases} 0 & \text{si } \mathbf{w}^T \mathbf{x} \leq \theta \\ 1 & \text{si } \mathbf{w}^T \mathbf{x} > \theta \end{cases} \quad (2.1)$$

La frontera de decisión $\mathbf{w}^T \mathbf{x} = \theta$ define un hiperplano separador en el que $\mathbf{w}$ fija su orientación y θ su desplazamiento. Introduciendo $b = -\theta$ y $z = \mathbf{w}^T \mathbf{x} + b$, la regla (2.1) se expresa con la función escalón de Heaviside (Figura 2.3):

$$\text{Salida} = H(z) = \begin{cases} 0 & \text{si } z \leq 0 \\ 1 & \text{si } z > 0 \end{cases} \quad (2.2)$$

2.3 Arquitectura de la red MLP

2.3.1 Topología *feedforward* y estructura de capas

En la sección anterior se ha detallado cómo un perceptrón simple actúa definiendo un único hiperplano separador en el espacio de características $\mathbb{R}^n$. Esta simplicidad impone una limitación fundamental en su capacidad de representación: el modelo solo es capaz de clasificar correctamente conjuntos de datos que sean linealmente separables. Si bien esto puede resultar suficiente para problemas elementales, en la inmensa mayoría de casos reales se busca modelar fronteras de decisión continuas, complejas y fuertemente no lineales, por lo que resulta imperativo combinar múltiples unidades computacionales de este tipo, dando lugar a la arquitectura conocida como Perceptrón Multicapa o MLP (*Multilayer Perceptron*).

Siguiendo la conceptualización expuesta por Nielsen [4], un MLP supera esta limitación organizando las neuronas en una estructura de múltiples niveles o capas interconectadas. Desde un punto de vista topológico, esta arquitectura se define como un grafo acíclico dirigido (DAG). A esta familia de modelos se la denomina Redes Prealimentadas (*feedforward*), da-

do que la información fluye en una única dirección, sin formar bucles. De esta manera, la información que recibe la primera capa se transforma secuencialmente capa por capa. Este proceso iterativo permite incrementar el grado de no linealidad del modelo, facilitando una emulación mucho más precisa de las fronteras de decisión objetivo (Figura 2.2).

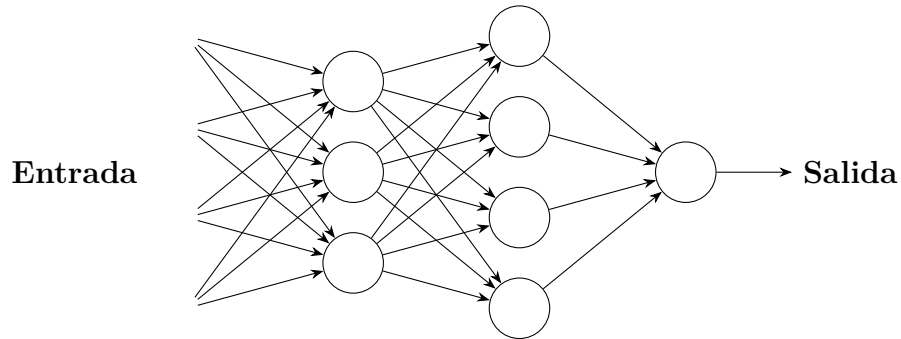


Figura 2.2: Arquitectura de una Red Neuronal Multicapa

Formalmente, una red neuronal MLP está estructurada en una secuencia de niveles discretos. Si denotamos por L al número total de capas de la red, podemos clasificar estas capas en tres tipologías distintas, indexadas por la variable $l \in \{0, 1, \dots, L\}$:

- **Capa de entrada** ($l = 0$): Es el punto de entrada de la información al sistema. A diferencia del resto de la red, en esta capa no se realiza ninguna transformación matemática; simplemente se distribuyen las características del problema. Matemáticamente se representa como un vector $\mathbf{x} \in \mathbb{R}^{n_0}$, donde n_0 corresponde a la dimensión n del espacio de características.
- **Capas ocultas** ($0 < l < L$): Constituyen el núcleo de procesamiento de la red. Un MLP puede contener múltiples capas ocultas, cuya profundidad y número de parámetros definen lo que se conoce como “*Deep Learning*”. Cada capa oculta l contiene n_l neuronas, las cuales reciben como entrada la salida de la capa inmediatamente anterior ($l - 1$), aplican una transformación afín y, posteriormente, una función de activación no lineal. En estas capas se genera la no linealidad buscada.
- **Capa de salida** ($l = L$): Es la última capa de la topología y se encarga de colapsar la información procesada en las capas ocultas para emitir la predicción final del modelo. La dimensión de esta capa viene determinada por la naturaleza del problema a resolver. Dado que el objetivo de este trabajo es la verificación de firmas (un problema de clasificación binaria entre “auténtica” y “falsificada”), esta capa estará compuesta por una única neurona ($n_L = 1$), cuya salida representará la probabilidad de pertenencia a una de las dos clases.

2.3.2 Formalización matricial del modelo

Para describir matemáticamente el flujo de información a través de la arquitectura del MLP, es necesario generalizar las ecuaciones del perceptrón simple empleando álgebra lineal. Esta formalización matricial no solo simplifica y compacta la notación, sino que constituye la base analítica sobre la cual se implementan las operaciones en los entornos computacionales modernos.

Siguiendo la convención notacional estándar [4], definimos los siguientes elementos para cualquier capa l de la red (donde $1 \leq l \leq L$):

- **Matriz de pesos ($W^{(l)}$):** Agrupa todos los pesos sinápticos que conectan la capa anterior ($l - 1$) con la capa actual (l). Matemáticamente, $W^{(l)} \in \mathcal{M}_{n_l \times n_{l-1}}(\mathbb{R})$. El elemento $w_{jk}^{(l)}$ representa el peso de la conexión que parte desde la k -ésima neurona de la capa ($l - 1$) hacia la j -ésima neurona de la capa l .
- **Vector de sesgos ($\mathbf{b}^{(l)}$):** Contiene los términos de sesgo de todas las neuronas de la capa actual. Se define como $\mathbf{b}^{(l)} = [b_1^{(l)}, b_2^{(l)}, \dots, b_{n_l}^{(l)}]^T \in \mathbb{R}^{n_l}$, donde $b_j^{(l)}$ es el sesgo correspondiente a la j -ésima neurona de la capa l .
- **Vector de activaciones ($\mathbf{a}^{(l)}$):** Representa las salidas producidas por las neuronas de la capa l , siendo $\mathbf{a}^{(l)} \in \mathbb{R}^{n_l}$. El caso base ($l = 0$) se corresponde con el vector de características original, es decir, $\mathbf{a}^{(0)} \equiv \mathbf{x} \in \mathbb{R}^{n_0}$.

Con esta estructura algebraica definida, el potencial de activación que recibe la j -ésima neurona de la capa l , denotado como $z_j^{(l)}$, se calcula como el producto escalar de los pesos incidentes por las activaciones de la capa anterior, más su sesgo respectivo:

$$z_j^{(l)} = \sum_{k=1}^{n_{l-1}} w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)} \quad (2.3)$$

Gracias a la disposición escogida de los índices en la matriz $W^{(l)}$ (donde el índice de fila j denota el destino y el índice de columna k denota el origen), podemos definir el vector de potenciales $\mathbf{z}^{(l)} \in \mathbb{R}^{n_l}$ de la siguiente manera:

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (2.4)$$

Sobre este vector resultante se aplicará la función de activación pertinente, proceso que se detallará al estudiar la propagación hacia delante.

2.4 Funciones de activación

Como se estableció en la sección anterior, la operación fundamental en una capa de una red neuronal es la transformación afín sobre sus entradas. Sin embargo, si la arquitectura se limitara exclusivamente a encadenar este tipo de transformaciones, la profundidad de la red perdería por completo su utilidad. Para demostrarlo, sea $\mathbf{x}$ el vector de entrada original. El potencial de la primera capa viene dado por:

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

Si la segunda capa recibe el vector $\mathbf{z}^{(1)}$ y aplica nuevamente una transformación lineal, su salida matemática sería:

$$\mathbf{z}^{(2)} = W^{(2)}\mathbf{z}^{(1)} + \mathbf{b}^{(2)} = (W^{(2)}W^{(1)})\mathbf{x} + (W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}) = W'\mathbf{x} + \mathbf{b}'$$

Es decir, esta capa de profundidad no ha aportado ningún cambio en la estructura de la salida. Por consiguiente, una red profunda puramente lineal no posee mayor capacidad de representación que un perceptrón simple [5]. Es por ello que para modelar geometrías complejas y fuertemente no lineales, es necesario romper la linealidad aplicando una función no lineal sobre el potencial $\mathbf{z}^{(l)}$ de cada capa.

El modelo del perceptrón clásico, basado en la función escalón de Heaviside $H(z)$ (2.2), presenta una barrera infranqueable para la optimización basada en gradientes. Al ser una función discontinua, su derivada es estrictamente nula en todo su dominio salvo en $z = 0$, donde no está definida:

$$H'(z) = 0, \quad \forall z \in \mathbb{R} \setminus \{0\} \quad (2.5)$$

Esto implica que el gradiente de la función de coste respecto a los pesos (véase la Sección 2.7) será un vector nulo en casi todas partes. Esto imposibilita la actualización iterativa de parámetros mediante métodos de descenso. Aunque empíricamente es posible entrenar arquitecturas de una sola capa con esta función usando la Regla Delta (véase el Capítulo 4), el entrenamiento de redes profundas mediante *Backpropagation* exige funciones estrictamente diferenciables. Para solventar esta limitación analítica, se emplean funciones de activación continuas. En el contexto de la clasificación binaria de este trabajo, la elección matemática óptima para la capa de salida es la función logística o sigmoide (Figura 2.3):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.6)$$

Esta función aproxima el escalón mediante una curva suave de clase $\mathcal{C}^\infty$. Al proyectar cualquier valor real al intervalo abierto $(0, 1)$, permite interpretar la salida de la neurona directamente como la probabilidad condicionada $P(C_1|\mathbf{x})$, donde C_1 es una clase dentro de la clasificación; es decir, la probabilidad de que el vector de entrada $\mathbf{x}$ pertenezca a la clase C_1 . En el caso concreto de la clasificación de firmas, se mide la probabilidad de que la firma sea auténtica.

Además de su justificación probabilística, la sigmoide posee una derivada computacionalmente muy eficiente, ya que se expresa en términos de su propia salida:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (2.7)$$

Esta autosemejanza simplifica drásticamente el cálculo del gradiente, pues solo requiere evaluar la activación ya computada de la neurona.

Finalmente, para las capas ocultas de modelos más profundos, la literatura moderna admite otras funciones entre las que destacan la tangente hiperbólica ($\tanh$) y la Unidad Lineal Rectificada (ReLU), definida como $f(z) = \max(0, z)$ (Figura 2.3).

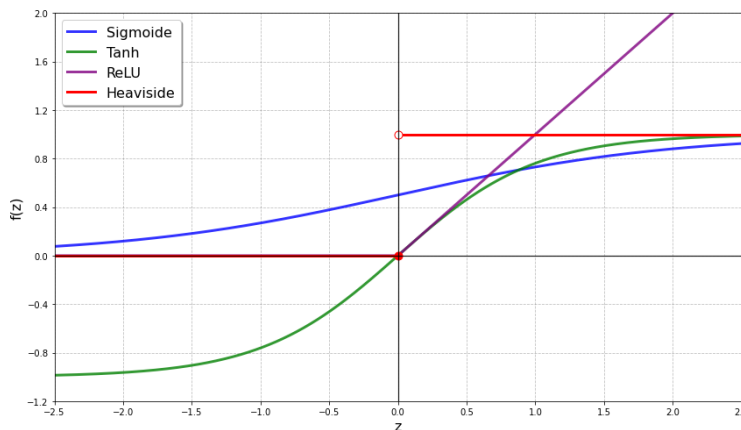


Figura 2.3: Funciones de activación ([*funciones_de_activacion.ipynb*](#))

2.5 Teorema de Aproximación Universal

Una vez establecida la necesidad de introducir no linealidades en el modelo mediante funciones de activación continuas, surge la pregunta de si existe algún límite en la capacidad de representación de esta arquitectura. La respuesta a esta cuestión la proporciona el Teorema de Aproximación Universal, demostrado rigurosamente por George Cybenko en 1989 [6].

Teorema 2.1 (Cybenko, 1989). Sea $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función continua y sigmoideal (es decir, $\sigma(z) \rightarrow 1$ cuando $z \rightarrow \infty$ y $\sigma(z) \rightarrow 0$ cuando $z \rightarrow -\infty$). Sea K un subconjunto compacto de $\mathbb{R}^n$ y sea $f : K \rightarrow \mathbb{R}$ una función continua arbitraria. Entonces, para cualquier $\epsilon > 0$, existe un entero $N > 0$, constantes reales $v_i, b_i \in \mathbb{R}$ y vectores de pesos $\mathbf{w}_i \in \mathbb{R}^n$ tales que la función $F(\mathbf{x})$ definida por la red neuronal:

$$F(\mathbf{x}) = \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i),$$

satisface la desigualdad:

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \quad \forall \mathbf{x} \in K.$$

Este resultado garantiza que una combinación lineal de sigmoides es capaz de aproximar cualquier función continua sobre un conjunto compacto. Aplicado al contexto de este trabajo, implica que existe una configuración del MLP capaz de separar firmas auténticas de falsificaciones con un error arbitrariamente pequeño.

No obstante, el teorema es puramente existencial; no indica cómo encontrar dichos pesos ni asegura que el entrenamiento converja a ellos. Además, aunque una única capa oculta suficientemente ancha bastaría en teoría, en la práctica es más eficiente realizar un aumento en la profundidad para extraer jerarquías de características, premisa fundamental que motiva el uso de CNN en el Capítulo 3.

2.6 Propagación hacia delante (*Forward Propagation*)

Una vez definidos los componentes topológicos de la red neuronal, es necesario formalizar el algoritmo mediante el cual el modelo procesa la información. Este proceso se conoce como propagación hacia delante (*Forward Propagation*) [5]. En una arquitectura *feedforward*, el flujo de datos es unidireccional. La información se propaga desde la capa de entrada hasta la capa de salida, evaluando distintas transformaciones anidadas, sin ciclos en el grafo computacional (Figura 2.2).

2.6.1 Inicialización: capa de entrada

Algorítmicamente, el proceso de propagación se puede definir como una secuencia recursiva. El caso base de esta recursión se establece en la capa de entrada, denotada como la capa 0 o capa visible. En esta capa, no se realiza ninguna transformación afín sobre los datos.

Sea $\mathbf{x} \in \mathbb{R}^{n_0}$ un vector columna de características de entrada; la inicialización del algoritmo se define estableciendo el vector de activación de la capa cero, $\mathbf{a}^{(0)}$, como el vector de entrada

original:

$$\mathbf{a}^{(0)} = \mathbf{x}$$

donde n_0 representa la dimensionalidad del vector de entrada (es decir, el número total de características o píxeles extraídos de la imagen).

2.6.2 Propagación en capas ocultas

Una vez inicializado $\mathbf{a}^{(0)}$, la red procesa la información secuencialmente. Para cualquier capa oculta l (donde $l = 1, 2, \dots, L - 1$), el cálculo de su vector de activaciones depende exclusivamente de la salida generada por la capa inmediatamente anterior, $\mathbf{a}^{(l-1)}$ [4]. Esta transformación se descompone en dos operaciones matemáticas ejecutadas una tras otra:

1. **Cálculo del potencial (transformación afín):** En primer lugar, se calcula $\mathbf{z}^{(l)}$ mediante la transformación vista en la Sección 2.2.2 (2.4):

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

2. **Cálculo de la activación (transformación no lineal):** A continuación, se aplica un vector de funciones de activación $\mathbf{f}^{(l)} = [f_1^{(l)}, f_2^{(l)}, \dots, f_{n_l}^{(l)}]^T$ (discutidas en la Sección 2.3), típicamente con activaciones idénticas ($f_i^{(l)} = f_j^{(l)}$). Este vector se aplica elemento a elemento sobre el vector de potencial para romper la linealidad del sistema y obtener el vector de activación final de la capa:

$$\mathbf{a}^{(l)} = \mathbf{f}^{(l)}(\mathbf{z}^{(l)})$$

Aplicando este par de ecuaciones de forma iterativa, el algoritmo es capaz de propagar la información a través de todas las capas ocultas, abstrayendo características cada vez más complejas en cada nivel de profundidad.

2.6.3 Capa de salida y predicción final

El proceso recursivo culmina en la última capa, denotada como la capa L . En esta etapa, la red consolida todas las características extraídas previamente para emitir un veredicto. Al igual que en las capas ocultas previas, el cómputo se inicia evaluando el potencial $z^{(L)}$. No obstante, la particularidad de esta etapa radica en la transformación no lineal aplicada.

Dado que el objetivo del modelo en estudio es la resolución de un problema de clasificación binaria, se aplica la función de activación logística o sigmoide σ , tal y como se fundamentó en la Sección 2.3. Por tanto, las ecuaciones que rigen el final de la propagación hacia delante

se definen como:

$$z^{(L)} = W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$$

$$\hat{y} = \mathbf{a}^{(L)} = \sigma(z^{(L)})$$

La variable $\hat{y}$ representa la salida final o predicción escalar de la red neuronal. Gracias a la compresión matemática ejercida por la función sigmoide, este valor está acotado estrictamente en el intervalo continuo $(0, 1)$. Desde una perspectiva estadística formal [7], la convergencia de esta salida permite interpretarla como la probabilidad condicionada $P(C_1 | \mathbf{x})$. En el contexto de este trabajo, $\hat{y}$ representa el nivel de confianza del modelo respecto a que el vector de características original $\mathbf{x}$ pertenezca a la clase positiva C_1 (firma auténtica). Para transformar esta predicción probabilística continua en una decisión discreta definitiva, se aplica un umbral de decisión escalar, denotado por τ (con un valor por defecto de 0,5). La clasificación binaria final se formaliza evaluando la función escalón de Heaviside en la expresión $H(\hat{y} - \tau)$. Si $\hat{y} > \tau$, el valor obtenido es 1 y la firma se clasifica como auténtica; en caso contrario, se obtiene 0 y se rechaza catalogándose como falsificación.

Finalmente, cabe destacar que, aunque esta formulación matemática se ha definido para una única observación $\mathbf{x}$, en la implementación computacional práctica resulta ineficiente procesar las imágenes de las firmas secuencialmente. Por ello, las ecuaciones se generalizan para procesar simultáneamente un lote (*batch*) de m firmas. Mediante un proceso de vectorización matricial, el vector de entrada se expande a una matriz de características $X \in \mathbb{R}^{n_0 \times m}$, transformando los vectores de estado intermedios en matrices $Z^{(l)}$ y $A^{(l)}$. De este modo, la salida final del algoritmo es un vector fila $\hat{Y} \in \mathbb{R}^{1 \times m}$, donde cada componente $\hat{y}_i$ contiene la predicción probabilística correspondiente a la i -ésima firma del lote evaluado.

2.7 Funciones de pérdida

Una vez emitida la predicción probabilística, la función de pérdida cuantifica la magnitud del error, midiendo la distancia entre la salida del modelo ($\hat{y}$) y la etiqueta real (y).

2.7.1 El Error Cuadrático Medio (MSE) y sus limitaciones

El Error Cuadrático Medio para una observación se define como:

$$\mathcal{L}_{MSE} = \frac{1}{2}(y - \hat{y})^2 \quad (2.8)$$

A pesar de ser la función natural en regresión, en clasificación binaria con sigmoides puede ralentizar el aprendizaje. Si la red predice con total confianza que una firma falsa es auténtica,

la derivada de la sigmoide tiende a cero y el gradiente apenas corrige los pesos [4].

2.7.2 Entropía Cruzada Binaria (BCE): Formulación y propiedades

En la clasificación binaria, la etiqueta sigue una distribución de Bernoulli, y la red estima la probabilidad condicionada $\hat{y} = P(y = 1 \mid \mathbf{x})$. Por tanto, la función de masa es:

$$P(y \mid \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y} \quad (2.9)$$

Maximizar la verosimilitud equivale a minimizar el logaritmo negativo de dicha función, obteniendo la Entropía Cruzada Binaria [7]:

$$\mathcal{L}_{BCE} = -\log P(y \mid \mathbf{x}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (2.10)$$

La interpretación intuitiva de la ecuación es la siguiente:

- **Si la firma es auténtica** ($y = 1$): Se obtiene $\mathcal{L} = -\log(\hat{y})$. Si la red predice correctamente y con alta confianza ($\hat{y} \rightarrow 1$), el logaritmo tiende a cero, por lo que el coste es nulo. Por otro lado, si la red predice que es falsa ($\hat{y} \rightarrow 0$), el logaritmo dispara la penalización hacia el infinito.
- **Si la firma es falsa** ($y = 0$): Se obtiene $\mathcal{L} = -\log(1 - \hat{y})$, y se aplica la misma lógica, pero en sentido inverso.

El uso de esta función de pérdida soluciona el problema del desvanecimiento del gradiente. Suponiendo la predicción $\hat{y} = \sigma(z)$, al derivar la función, se obtiene la expresión:

$$\frac{\partial \mathcal{L}_{BCE}}{\partial z} = \hat{y} - y \quad (2.11)$$

Esta expresión demuestra que el gradiente de la pérdida es directamente proporcional al error cometido ($\hat{y} - y$), evitando el desvanecimiento asociado al MSE con sigmoide.

3. Fundamentos Matemáticos de las Redes Convolucionales (CNN)

3.1 Limitaciones del MLP en el procesamiento de imágenes

A pesar de que el MLP puede aproximar funciones continuas, presenta varias limitaciones claras al procesar imágenes. Primero, exige aplanar la matriz de píxeles en un vector (*flattening*), destruyendo la estructura bidimensional y la noción de adyacencia espacial de los píxeles. Segundo, las capas densas disparan el número de parámetros: una imagen de 200×200 píxeles conectada a 1000 neuronas requeriría un total de 4×10^7 pesos, favoreciendo el sobreajuste por tener una mayor cantidad de parámetros que de datos, y dificultando la generalización ante desplazamientos o rotaciones. Las Redes Neuronales Convolucionales resuelven estos problemas sustituyendo la conectividad total por operaciones locales de álgebra tensorial.

3.2 La operación de convolución discreta en 2D

3.2.1 Matrices de Entrada y *Kernel*

Contrariamente al procesamiento vectorial del MLP, las capas convolucionales preservan la topología bidimensional de los datos. Para ello, es necesario redefinir las estructuras que intervienen en el modelo. La entrada al modelo se define como una matriz de intensidades de píxeles, denotada como $I \in \mathbb{R}^{H \times W}$, donde H y W denotan la altura y anchura de la imagen, respectivamente. En el caso de este trabajo, al trabajar con imágenes en escala de grises, la profundidad del canal de color es de dimensión 1. Esto nos permite tratar directamente con matrices bidimensionales, sin necesidad de recurrir a tensores tridimensionales, como en el caso de las imágenes RGB.

Por otro lado, la matriz de pesos sinápticos que conecta las neuronas en el MLP es reemplazada por una estructura más compacta llamada filtro o *kernel*. Este se define como una matriz de parámetros ajustables $K \in \mathbb{R}^{f \times f}$, con $f \ll \min\{H, W\}$. El valor de f suele ser impar, lo que garantiza la existencia de un elemento central en la matriz del filtro. El objetivo del *kernel* no es procesar globalmente la imagen, sino detectar características locales.

Mediante el entrenamiento, cada matriz K optimiza sus valores operando sobre I para detectar patrones visuales específicos, como puede ser la inclinación de un trazo, un bucle, una intersección o cualquier otro patrón abstracto.

3.2.2 Operación de convolución

La dinámica de una capa convolucional consiste en deslizar el filtro a lo largo de toda la superficie de la matriz de entrada I . En cada una de las posiciones, se calcula el producto escalar entre los valores del filtro K y las intensidades de los píxeles que caen bajo la región de cobertura geométrica del *kernel*, para posteriormente sumarlos y obtener un único valor. Formalmente, el valor en la posición (i, j) de la matriz resultante, denotada como $S \in \mathcal{M}_{O \times O}(\mathbb{R})$, donde O es la dimensión de la matriz resultante, se define mediante la ecuación de la convolución discreta bidimensional [8]:

$$S(i, j) = (I * K)(i, j) = \sum_{m=1}^f \sum_{n=1}^f I(i + m - 1, j + n - 1) K(m, n) \quad (3.1)$$

Esta ecuación utiliza un anclaje superior izquierdo, es decir, a partir del píxel (i, j) , los píxeles sobre los que se opera en la convolución son los de su derecha y los de debajo.

La expresión (3.1) se puede adaptar a un anclaje centrado (Figura 3.1). Suponiendo f impar y definiendo el radio $k = (f - 1)/2$, resulta la siguiente fórmula:

$$S(i, j) = \sum_{m=1}^{2k+1} \sum_{n=1}^{2k+1} I(i + m - k - 1, j + n - k - 1) K(m, n) \quad (3.2)$$

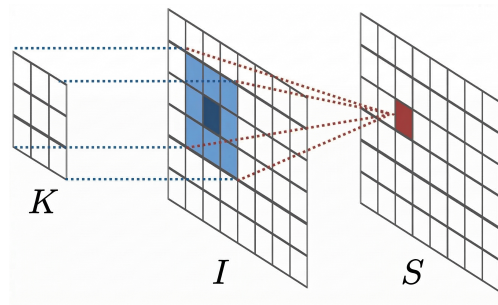


Figura 3.1: Representación del proceso de convolución con anclaje centrado [9]

3.2.3 Hiperparámetros geométricos (*Padding* y *Stride*)

Tal y como se han definido las operaciones convolucionales, surge una pregunta lógica: ¿Qué ocurre en los bordes de la imagen donde el filtro excede los límites espaciales?

Para responder a la pregunta anterior, es necesario introducir dos nuevos hiperparámetros. Si se aplica el filtro K requiriendo que se encuentre íntegramente dentro de la imagen I (convolución válida), los píxeles de los bordes se operan menos veces que los centrales, y la matriz resultante sufre una reducción de tamaño en cada capa. Para solucionarlo, se introduce el relleno o *padding*, denotado como p . Esta técnica consiste en añadir un margen artificial de píxeles rodeando la imagen original. Una de las estrategias más universales es el relleno de ceros [10], lo que permite centrarse en los píxeles de los extremos sin que el sumatorio colapse. El parámetro p representa el número de filas (y columnas) que se añaden a cada borde. Si se pretende conservar el tamaño de la imagen convolucionada, se debe tomar el valor $p = (f - 1)/2$.

El segundo hiperparámetro es el paso o *stride*, denotado como s . Este escalar determina el tamaño del salto que se realiza tras computar cada operación. Si $s = 1$, entonces se tiene un paso continuo, evaluando regiones píxel a píxel. Por el contrario, si $s > 1$, el filtro omitirá píxeles intermedios hasta volver a operar, lo que disminuye la dimensión de la matriz resultante. Suponiendo una matriz de entrada y un filtro cuadrados, el tamaño resultante de dicha matriz se calcula mediante la siguiente expresión [8]:

$$O = \left\lfloor \frac{W - f + 2p}{s} \right\rfloor + 1 \quad (3.3)$$

3.2.4 Mapa de Características (*Feature Map*) y Pesos Compartidos

El resultado de aplicar la convolución sobre toda la imagen I es la matriz de salida S , conocida como mapa de características o *feature map* (Figura 3.3). Este tensor codifica la intensidad de un cierto patrón específico que el filtro K ha aprendido a detectar. A diferencia de un MLP, donde cada píxel tiene asociado un peso distinto y exclusivo, la capa convolucional utiliza los pesos de la matriz K para evaluar todas las posiciones de la imagen. Esto se conoce como “pesos compartidos” y resuelve las dos grandes limitaciones del MLP.

En primer lugar, la complejidad paramétrica del modelo se reduce, pues las conexiones en los píxeles desaparecen y solo es necesario el aprendizaje de los valores de un pequeño filtro, mitigando el riesgo de sobreajuste. En segundo lugar, los pesos compartidos permiten detectar las mismas características aunque aparezcan trasladadas sobre la imagen. Al desplazar un mismo trazo a otra coordenada, el filtro sigue detectando la característica con la misma eficacia, reflejándose desplazada dentro del mapa de características S .

3.3 Capas de agrupamiento y reducción dimensional

3.3.1 Mecánica geométrica y tipos de agrupamientos (*Pooling*)

Para evitar el disparo en el número de parámetros al pasar a capas densas, las capas de convolución se intercalan con capas de agrupamiento, que reducen la dimensión de los mapas de características. Usan una ventana $f \times f$ y un paso s , pero no tienen pesos entrenables [8]. Existen dos funciones de agrupamiento principales: el *Average-Pooling*, que promedia los valores de la región, y el *Max-Pooling*, que conserva el máximo y suele ser preferible porque mantiene activaciones fuertes asociadas a trazos relevantes (Figura 3.3).

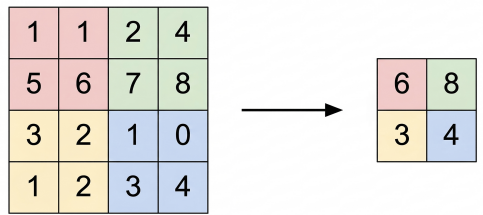


Figura 3.2: *Max-Pooling* 2×2 con salto $s = 2$ [11]

3.3.2 Reducción dimensional e invarianza espacial

El uso intercalado de capas de agrupamiento viene motivado por dos propósitos. El primero, y más evidente, es que la capa actúa como un mecanismo de reducción dimensional. Llamando D a la dimensión de la matriz de salida, suponiendo una matriz de entrada cuadrada de tamaño $W \times W$, la dimensión de salida se rige por la misma expresión que la convolución (3.3), asumiendo un relleno nulo ($p = 0$):

$$D = \left\lfloor \frac{W - f}{s} \right\rfloor + 1 \quad (3.4)$$

En la práctica, el *Max-Pooling* estándar es una ventana de tamaño $f = 2$ con un paso $s = 2$ (Figura 3.2). Este agrupamiento provoca una reducción de tamaño a la mitad en cada eje, por lo que el área de la matriz de características se reduce a la cuarta parte. Esta compresión minimiza el coste computacional en las capas densas posteriores y mitiga el riesgo de sobreajuste.

En segundo lugar, estas capas otorgan al modelo invarianza a la traslación local. Al extraer el máximo valor de cada región, pierde importancia la posición exacta del píxel que activa la capa. Además, en la práctica se aplican varios agrupamientos, por lo que en el caso del procesamiento de imágenes, no importa si una firma está desplazada unos milímetros en el lienzo;

las capas de agrupamiento serán capaces de detectar igualmente las características.

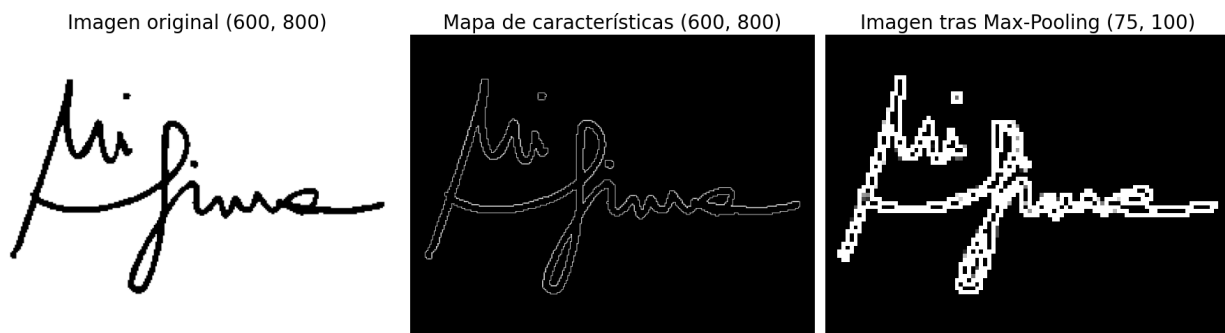


Figura 3.3: Ejemplo de convolución con un núcleo convolucional de 3×3 para la detección de bordes, en el que todas las entradas de la matriz valen -1 salvo la central que vale 8 ($f = 3$, $s = 1$, $p = 1$) y un *Max-Pooling* que reduce el tamaño 64 veces ($f = 8$, $s = 8$) ([ejemplo_de_convolucion.ipynb](#)).

3.4 Arquitectura general de una CNN

Hasta ahora, se ha analizado de forma aislada cada uno de los componentes que intervienen en una CNN. Sin embargo, el verdadero potencial de este tipo de redes reside en la combinación de estos elementos, distinguiendo dos etapas en el procesamiento: extracción de características y clasificación (Figura 3.4).

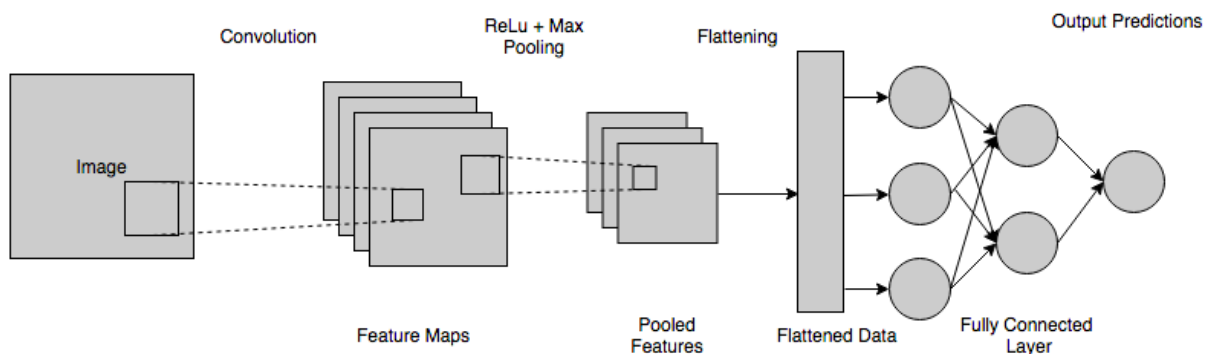


Figura 3.4: Estructura general de una CNN [12]

3.4.1 Fase de extracción de características

El objetivo en la primera etapa es representar de manera abstracta la matriz de píxeles de entrada. Para lograrlo, se recurre a la concatenación de múltiples bloques de procesamiento, formados por tres operaciones consecutivas: una convolución discreta, una función no lineal (ReLU) y un agrupamiento (*Max-Pooling*). Debido a esto, la red adquiere una capacidad

de aprendizaje jerárquico de características [5]. En el primer bloque, los filtros se aplican directamente sobre la imagen original, por lo que estas capas detectan patrones visuales muy básicos, como bordes o direcciones sencillas.

A medida que los datos avanzan hacia bloques más profundos, la dimensión espacial de los mapas de características disminuye drásticamente gracias al agrupamiento. No obstante, el número de filtros suele incrementarse, debido a que la compresión del mapa de características permite que los filtros abarquen regiones más extensas de la imagen original, lo que permite reconocer patrones complejos como intersecciones o bucles. El resultado final de esta fase es un tensor tridimensional espacialmente pequeño, pero con una gran profundidad de canales resultantes de los diferentes filtros aplicados, eliminando por completo píxeles irrelevantes como los del fondo de la imagen.

3.4.2 Fase de transición y clasificación

Al finalizar la etapa de extracción, se obtiene un conjunto de matrices bidimensionales que contienen las características de la imagen. Sin embargo, para poder procesar esta información como en un MLP, es necesario transformar el tensor tridimensional de características en un vector unidimensional, proceso conocido como aplanamiento o *flattening*. Esta operación reorganiza los elementos de las matrices concatenando sus filas hasta obtener un único vector $\mathbf{x} \in \mathbb{R}^n$. En este punto se descarta la topología bidimensional.

A continuación, con la información vectorizada, se propaga hacia capas densas, utilizando una arquitectura análoga a la del MLP analizado en el Capítulo 2. El objetivo ahora no es buscar patrones en los píxeles, sino aprender a combinar las características obtenidas, con el fin de establecer una frontera de decisión. En el contexto particular de la clasificación binaria, en la última capa, formada únicamente por una neurona, se aplica la función sigmoide, lo que permite emitir una predicción $\hat{y} \in (0, 1)$ que representa la probabilidad de pertenecer a una clase.

4. Algoritmos de Optimización y Entrenamiento

4.1 El algoritmo de Backpropagation

4.1.1 Objetivo matemático y la Regla de la Cadena

El algoritmo de retropropagación de errores, *Backpropagation*, es el núcleo computacional del aprendizaje profundo. Antes de profundizar en su funcionamiento, es importante recalcar que este algoritmo no actualiza ningún parámetro de la red neuronal. Su único propósito es el cálculo eficiente de las derivadas parciales de la función de pérdida $\mathcal{L}$ con respecto a cada parámetro ajustable del modelo, ya sean conexiones en las capas densas o valores individuales de los filtros [5]. Matemáticamente, el objetivo es aislar cómo una perturbación infinitesimal en un parámetro específico w afecta al error final de la predicción. Es decir, consiste en calcular la derivada parcial $\frac{\partial \mathcal{L}}{\partial w}$ para todos los elementos de la arquitectura.

El verdadero desafío analítico radica en que una red neuronal profunda es una gigantesca función matemática compuesta: la salida de la primera capa actúa como entrada de la segunda, cuya salida alimenta a la tercera, y así sucesivamente hasta llegar a la predicción final, donde se evalúa el error mediante la función de pérdida. Debido a este anidamiento de transformaciones no lineales, no se puede calcular directamente la derivada del error respecto a los pesos de las primeras capas. Es por ello que el algoritmo recurre al teorema de la regla de la cadena. Esta propiedad establece que la derivada de una función compuesta se puede calcular multiplicando las derivadas de sus funciones intermedias. En el caso de tres variables dependientes x , y y z , la regla se define como:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \quad (4.1)$$

En el contexto de las redes neuronales, esta regla permite que la señal del error se propague hacia atrás. Si se conoce la derivada del error con respecto a la salida de una capa, se puede multiplicar por la derivada interna de esa misma capa para averiguar la contribución al error de los pesos de la capa. Aplicando este principio de forma recursiva desde la predicción final $\hat{y}$ hasta la imagen de entrada, el algoritmo consigue calcular la contribución exacta de cada

parámetro al error global, construyendo de este modo el vector gradiente.

4.1.2 Ciclo de evaluación: *Forward y Backward Pass*

En la práctica computacional, la ejecución del algoritmo se estructura en dos fases complementarias. La primera fase se denomina propagación hacia delante o *forward pass*. En esta etapa, la información de la matriz de la imagen original atraviesa todas las capas de la arquitectura, desde los bloques de extracción de características hasta las capas profundas del MLP. Los filtros y las neuronas calculan sus activaciones de forma secuencial, como se detalló en los Capítulos 2 y 3, hasta que la capa de salida emite la predicción final $\hat{y}$. Inmediatamente después, se evalúa la función de pérdida, típicamente Entropía Cruzada Binaria, comparando la predicción con la etiqueta real y , lo que cuantifica el error escalar cometido en esa iteración.

Una vez obtenido el error, comienza la segunda fase: la propagación hacia atrás o *backward pass*. Partiendo del valor de la función de pérdida y aplicando la regla de la cadena, el sistema calcula en primer lugar las derivadas de la última capa. A continuación, estas magnitudes se reutilizan iterativamente para calcular las derivadas de la capa oculta inmediatamente anterior, de modo que la señal del error se propaga en sentido inverso a través de todas las operaciones hasta alcanzar los filtros iniciales de la red. Al concluir esta fase, el algoritmo devuelve el vector gradiente completo, denotado por $\nabla \mathcal{L}$, el cual agrupa las derivadas parciales de todos los parámetros ajustables de la red y posee una propiedad matemática fundamental: apunta en la dirección geométrica de máximo crecimiento del error. Por tanto, este vector es esencial para la actualización de pesos utilizada por los algoritmos de optimización, dando lugar al aprendizaje efectivo del modelo.

4.2 La regla Delta y la regla Delta Generalizada

Una vez calculado de manera eficiente el vector gradiente, la red necesita un mecanismo para ajustar sus parámetros minimizando el error. El fundamento matemático tiene su origen en la denominada Regla Delta, formulada por Widrow y Hoff [13]. Esta regla, enfocada en arquitecturas de una sola capa, establece que el cambio que se debe aplicar a un peso sináptico debe ser directamente proporcional al error cometido por la neurona y a la magnitud de entrada transmitida en dicho peso. Matemáticamente se expresa de la siguiente forma:

$$w_i^{(t+1)} = w_i^{(t)} + \Delta w_i^{(t)}, \quad \text{con} \quad \Delta w_i^{(t)} = \eta(y - \hat{y}^{(t)})x_i^{(t)}, \quad (4.2)$$

donde $(y - \hat{y})$ representa el error directo entre la etiqueta real y la predicción, x_i es la activa-

ción de la entrada y η es la tasa de aprendizaje (*learning rate*), que controla la magnitud del paso de actualización (suele estar entre 10^{-1} y 10^{-4}). Es importante configurar correctamente esta tasa, pues valores demasiado altos provocarán oscilaciones que impidan la convergencia al mínimo, mientras que valores muy bajos resultarán en un aprendizaje prácticamente nulo.

Sin embargo, esta formulación presenta una limitación insalvable para las redes profundas: exige conocer explícitamente la salida deseada y para cada neurona, pero en las capas ocultas no existe una etiqueta objetivo. Fue entonces cuando Rumelhart, Hinton y Williams formularon la Regla Delta Generalizada [14]. Este avance sustituye la diferencia $(y - \hat{y})$ por el gradiente de la función de pérdida. Apoyándose en el teorema de la regla de la cadena, la regla generalizada utiliza la derivada parcial para actualizar los pesos:

$$w_i^{(t+1)} = w_i^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial w_i^{(t)}} \quad (4.3)$$

4.3 Descenso de Gradiente Estocástico

En la formulación original del Descenso de Gradiente, el gradiente $\nabla \mathcal{L}$ debe calcularse como el promedio de los gradientes individuales de todos los ejemplos del conjunto de datos de entrenamiento. Matemáticamente, para un conjunto de N datos (en este caso, firmas), el gradiente esperado se define como:

$$\nabla \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla \mathcal{L}_i \quad (4.4)$$

Los conjuntos de datos actuales contienen miles de imágenes de alta resolución, por lo que calcular la suma exacta en cada iteración requeriría una cantidad de memoria RAM y un tiempo inasumibles. Es por ello que se introduce el Descenso de Gradiente Estocástico, o *SGD* por sus siglas en inglés. La versión más pura de este algoritmo aproxima el gradiente total calculando la derivada de la función de pérdida utilizando únicamente una muestra aleatoria ($m = 1$) en cada paso.

Al usar un único ejemplo, se acelera drásticamente el cálculo de actualizaciones, pero a su vez aumenta la cantidad de ruido, lo que puede provocar oscilaciones violentas en la minimización. Para solventarlo, se utiliza la variante de SGD por mini-lotes. Este enfoque utiliza un número reducido de muestras aleatorias, típicamente $m \in [16, 256]$ [14]:

$$\nabla \mathcal{L} \approx \frac{1}{m} \sum_{i=1}^m \nabla \mathcal{L}_{k(i)}, \quad (4.5)$$

donde $k(i)$ denota el índice aleatorio de cada muestra extraída del conjunto de datos. En cada época, el algoritmo realiza diferentes iteraciones utilizando distintos mini-lotes y actualizando los pesos siguiendo la ecuación (4.3) hasta haber procesado al menos una vez cada imagen del conjunto de datos. Al promediar el error sobre m firmas en cada iteración, se reduce la varianza del gradiente, lo que estabiliza el descenso y la convergencia. Por otro lado, existen bibliotecas de aprendizaje profundo que procesan los mini-lotes de forma simultánea mediante operaciones matriciales, lo que mejora la eficiencia computacional. Además, las pequeñas imprecisiones generadas al utilizar una muestra pequeña permiten a la red escapar de mínimos locales, facilitando la búsqueda del mínimo global.

4.4 Optimización avanzada: Estimación de Momentos Adaptativos (Adam)

A pesar de que el Descenso de Gradiente Estocástico por mini-lotes es altamente eficiente, presenta una limitación fundamental: aplica la misma tasa de aprendizaje (η) a todos los parámetros de la red. En la práctica, ciertos pesos requieren actualizaciones más amplias que otros, mientras que otros necesitan ajustes más finos. Es por ello que Kingma y Ba propusieron el algoritmo Adam (*Adaptive Moment Estimation*) [15]. Este método combina la base del descenso de gradiente con heurísticas de dos algoritmos de optimización previos, que se calculan en cada iteración para cada gradiente g_t (donde $g_t = \frac{\partial \mathcal{L}}{\partial w_t}$):

- **Momentum (Primer momento):** Emula la inercia de una bola rodando por una colina. El algoritmo acumula una media de los gradientes pasados que permite sortear mínimos locales y acelerar la convergencia. Matemáticamente se define como:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4.6)$$

- **RMSProp (Segundo momento):** Adapta la tasa de aprendizaje para cada peso mediante el cálculo de una media de los gradientes al cuadrado. Si un peso ha sufrido oscilaciones bruscas, la tasa disminuye, mientras que, si apenas se ha modificado, su tasa aumenta. La expresión matemática es:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4.7)$$

Los hiperparámetros β_1 y β_2 controlan las tasas de decaimiento de cada medida, y se establecen por defecto en 0,9 y 0,999, respectivamente. Dado que los vectores m_t y v_t se inicializan con ceros, sus estimaciones están sesgadas hacia cero en las primeras iteraciones, por lo que

se implementa una corrección de sesgo, obteniendo $\hat{m}_t$ y $\hat{v}_t$:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (4.8)$$

El algoritmo sustituye el gradiente por la inercia corregida $\hat{m}_t$ para decidir la dirección de descenso, y escala la tasa de aprendizaje base (η) dividiéndola por la raíz de la varianza corregida ($\sqrt{\hat{v}_t}$) más un término ϵ (típicamente 10^{-8}) introducido para evitar una división por cero:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (4.9)$$

A continuación, se presenta una comparativa entre los diferentes métodos de descenso mencionados mediante un ajuste de una recta de regresión bidimensional mal condicionada. Los resultados del valor de la función de pérdida se expresan en la Figura 4.1.

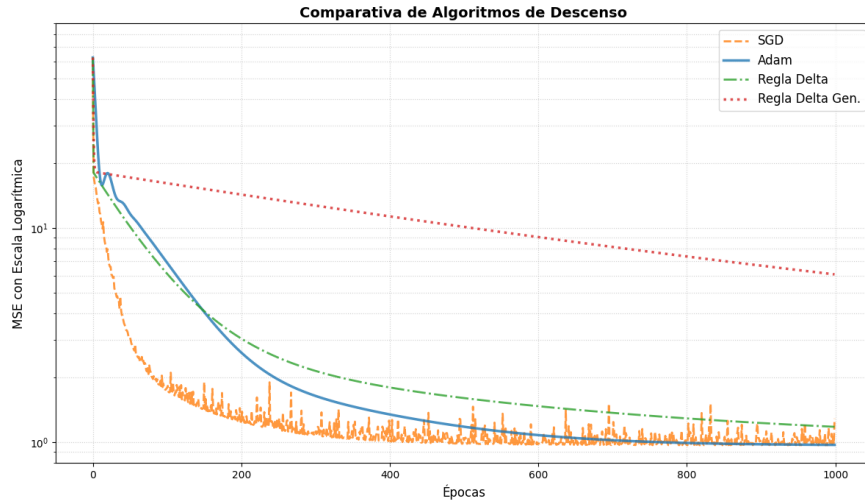


Figura 4.1: Comparativa entre algoritmos de descenso mediante un problema de regresión lineal bidimensional mal condicionada. Los datos han sido generados aleatoriamente, el primero entre 0 y 2, y el segundo entre 0 y 40. La variable objetivo se ha calculado utilizando una recta establecida añadiendo ruido. La red utiliza una única neurona con activación lineal ([comparativa_de_algoritmos_de_descenso.ipynb](#)).

Se observa que la Regla Delta muestra un descenso constante y suave, pero carece de velocidad. Por otro lado, la Regla Delta Generalizada, al computar todos los puntos, obtiene dos gradientes muy dispares, lo que, al utilizar una misma tasa de aprendizaje, ralentiza notablemente la actualización del peso de menor gradiente. El Descenso de Gradiente Estocástico muestra una alta volatilidad debido a la aleatoriedad de las muestras escogidas en cada iteración. Por último, se aprecia cómo el algoritmo de Adam tiene un rendimiento suave

y eficiente, logrando una convergencia rápida y sin ruido.

4.5 Regularización matemática: Control del sobreajuste

El objetivo de cualquier modelo de aprendizaje automático no es solo minimizar el error en los datos de entrenamiento, sino desarrollar la capacidad de generalizar ante datos nuevos. Cuando una red profunda posee una gran cantidad de parámetros ajustables, se vuelve susceptible a sufrir cierto sobreajuste, es decir, el modelo memoriza el ruido y las imperfecciones de imágenes de entrenamiento en lugar de aprender los patrones geométricos representativos. Como resultado, el error de entrenamiento desciende, mientras que el error en el conjunto de validación aumenta drásticamente, provocando un fracaso en la clasificación. Para mitigar esto, se utilizan distintas técnicas de regularización.

4.5.1 Penalización L2 (*Weight Decay*)

Una de las estrategias más extendidas para regularizar una red es la penalización L2 o *Weight Decay* [16], que restringe los valores numéricos que pueden alcanzar los pesos de la red, para evitar un crecimiento desproporcionado que conlleve una dependencia excesiva de un peso. Para ello, se modifica la función de pérdida original ($\mathcal{L}_0$) añadiendo un término proporcional al sumatorio de los cuadrados de todos los pesos de la red, resultando la siguiente función de pérdida:

$$\mathcal{L}_{total} = \mathcal{L}_0 + \frac{\lambda}{2} \sum_w w^2 \quad (4.10)$$

El hiperparámetro λ representa el factor de regularización; cuanto más cercano a cero, menor es la penalización. Por el contrario, un valor elevado fuerza a la red a mantener pesos cercanos a cero, lo que obliga al modelo a distribuir la toma de decisiones a través de los distintos filtros y características. El término $\frac{1}{2}$ se añade por conveniencia al derivar la función.

4.5.2 Regularización arquitectónica (*Dropout*)

Mientras que la penalización L2 controla la magnitud de los pesos, el algoritmo *Dropout* (desconexión aleatoria) modifica la estructura dinámica de la red [17], ignorando temporalmente un porcentaje de neuronas durante cada iteración de entrenamiento. A cada neurona se le asigna una probabilidad p de ser retenida (típicamente $p = 0,5$ en capas densas). De este modo, cada neurona se ve obligada a aprender características sin depender de un nodo específico. Este algoritmo solo se aplica en el entrenamiento.

5. Estudio práctico de firmas manuscritas

5.1 Metodología y generación del *Dataset*

El objetivo práctico del proyecto es el desarrollo de un sistema capaz de discriminar entre firmas auténticas y falsificadas utilizando arquitecturas de aprendizaje profundo. Para ello, es necesario un conjunto de datos sobre el que trabajar, por lo que se ha optado por diseñar, recopilar y procesar un *dataset* propio. Para estandarizar la recolección, se diseñó una plantilla en LaTeX, estructurada con 12 recuadros destinados a alojar cada firma (Figura 5.1)[[plantilla](#)].

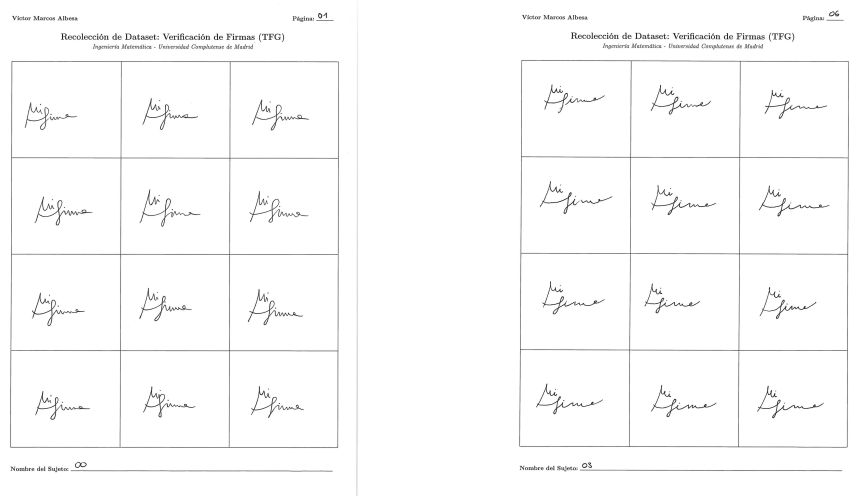


Figura 5.1: Ejemplo de plantilla rellena con firmas auténticas (izquierda) y falsificadas (derecha).

El proceso de recolección se dividió en dos fases:

- **Firmas auténticas:** El autor de este trabajo rellenoó un total de 63 hojas, generando 756 muestras de la firma auténtica a lo largo de varios meses. Por cuestiones de protección de datos, se ha inventado una firma específica para el proyecto (Figura 1.1), para no utilizar la firma personal del autor.

- **Firmas falsificadas:** Se solicitó la colaboración de 63 voluntarios. Cada persona rellenoó una hoja de firmas, proporcionando 12 falsificaciones por participante. Se decidió utilizar muestras de diferentes personas con el objetivo de aumentar la variedad de falsificaciones.

En total, el proceso de recolección generó 1512 firmas, divididas entre auténticas y falsificadas a partes iguales, logrando un conjunto de datos balanceado. Una vez digitalizadas las hojas en PDF, a escala de grises y con dimensiones 3507×2480 píxeles, se extrajo cada una de las imágenes mediante un *script* de Python ([extraccion_imagenes_dataset.ipynb](#)), lo que permite exportar todas las fotografías con las mismas dimensiones 320×240 píxeles.

5.2 Preparación de los datos

Gracias al uso de plantillas en la recolección, todas las imágenes poseen dimensiones uniformes de 320×240 . No obstante, para garantizar que las transformaciones posteriores no recorten extremos del trazo, se aplica un *padding* que añade 40 píxeles negros en los ejes superior e inferior, resultando imágenes cuadradas. Las imágenes se procesan en escala de grises, que toman valores entre 0 y 255, por lo que se realiza una normalización dividiendo entre 255 cada píxel, proyectando la matriz al rango $x \in [0, 1]$.

Para garantizar la viabilidad del modelo desarrollado, es necesario evaluarlo frente a datos no observados. Entrenar y evaluar una red sobre el mismo conjunto de imágenes conduciría a un sobreajuste masivo. Es por esto que las 1512 firmas se han dividido en tres subconjuntos disjuntos de forma aleatoria:

- **Conjunto de Entrenamiento (70 %):** Compuesto inicialmente por 1058 imágenes (529 auténticas y 529 falsificadas), constituye el núcleo de aprendizaje mediante el cual el modelo actualiza sus pesos iterativamente. Para mitigar el sobreajuste y ganar invariancia espacial, se aplica una técnica de aumento de datos sobre las 1058 imágenes de entrenamiento. Para cada una de ellas se generan 4 variaciones modificadas mediante transformaciones afines: rotaciones, traslaciones y reescalado. De este modo, se aumenta significativamente el número de imágenes de entrenamiento, pasando de 1058 muestras a 5290 (2645 auténticas y 2645 falsificadas).
- **Conjunto de Validación (15 %):** Compuesto por 226 imágenes (113 auténticas y 113 falsificadas), se emplea durante el entrenamiento como métrica de control para diagnosticar sobreajustes o actualizar la arquitectura de la red.
- **Conjunto de prueba (15 %):** Compuesto por 228 imágenes (114 auténticas y 114 falsificadas), se aísla en el desarrollo del modelo y reporta el rendimiento definitivo del

clasificador.

5.3 Diseño de las arquitecturas de red

5.3.1 Estructura del Perceptrón Multicapa

El Perceptrón Multicapa implementado tiene una topología en forma de embudo (Figura 5.2), lo que obliga a la red a comprimir la información y a extraer características representativas. Para optimizar la carga computacional y reducir el sobreajuste, las imágenes de entrada se han redimensionado a matrices de 64×64 . La matriz se aplana en la capa de entrada, obteniendo un vector de 4096 componentes. Las capas ocultas están compuestas por 512, 128 y 32 neuronas con función de activación ReLU. La última capa proyecta las 32 características restantes en una única neurona con función de activación sigmoide, que determina la probabilidad de pertenecer a cada clase. Por último, una neurona con función de activación Heaviside, con parámetro τ ajustable (por defecto 0,5), realiza la clasificación final (firma auténtica o falsificada).

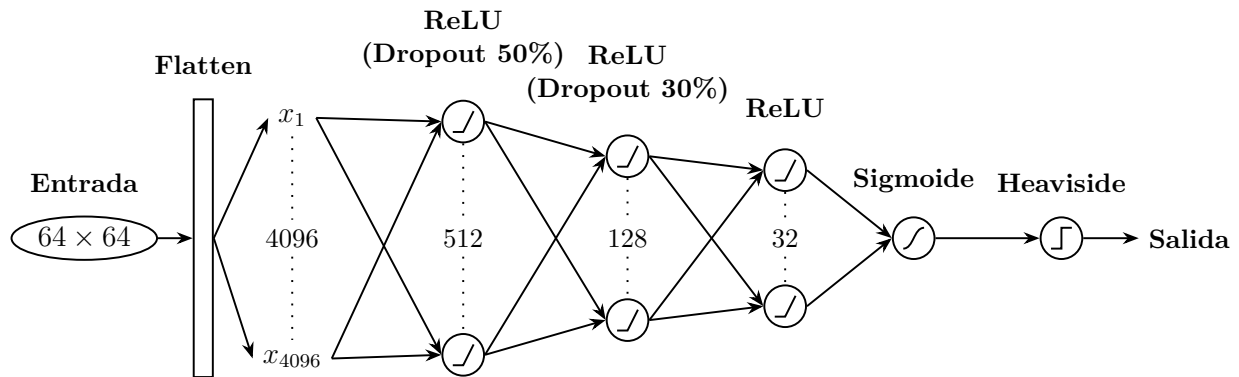


Figura 5.2: Arquitectura de la Red Neuronal Perceptrón Multicapa (MLP) propuesta.

5.3.2 Estructura de la Red Neuronal Convolutiva

Se ha diseñado una CNN para estudiar la capacidad de detección de geometrías locales de los trazos. La red se estructura en dos grandes secciones: un extractor de características y un clasificador final (Figura 5.3).

La fase de extracción de características recibe como entrada la imagen de tamaño 320×320 . La información pasa por un primer bloque que aplica 16 filtros convolucionales de tamaño 3×3 , con un relleno (*padding*) de 1 píxel para mantener los bordes. A continuación, se aplica una activación ReLU, y una capa de *Max-Pooling* 2×2 , reduciendo la imagen a 16 mapas de características de tamaño 160×160 . Le sigue un segundo bloque convolutiva idéntico

que reduce el tamaño a 80×80 . A continuación se aplica un tercer bloque compuesto por 32 filtros, que reduce nuevamente el tamaño de los mapas de características, y por último se aplica un cuarto bloque convolucional idéntico al anterior que emite 32 mapas de tamaño 20×20 .

La fase de clasificación final comienza con una capa de aplanamiento, donde se concatenan las características de los 32 mapas de características resultantes, obteniendo un vector de 12800 variables. Se aplica una única capa densa de 256 neuronas con activación ReLU, y a continuación se unifican las características restantes en una neurona con función de activación sigmoide, y una neurona con función de activación Heaviside emite la predicción final.

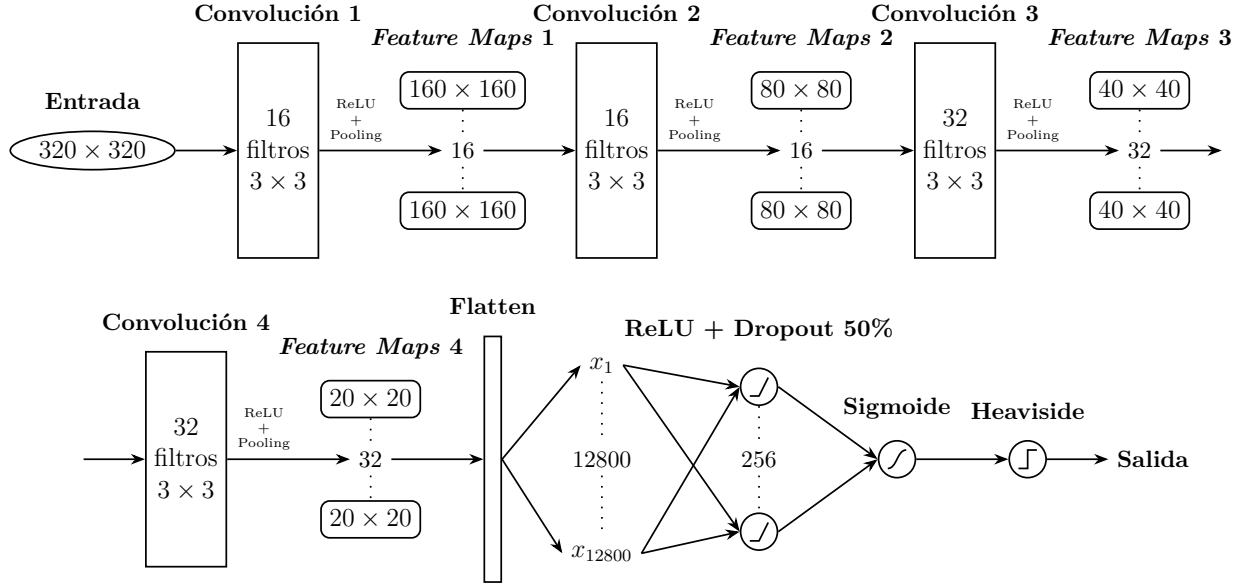


Figura 5.3: Arquitectura de la Red Neuronal Convolucional (CNN) propuesta.

5.4 Diseño experimental e implementación

Se han realizado un total de cuatro pruebas para cada tipo de red con el objetivo de evaluar su rendimiento una frente a otra y comparar la eficiencia de distintos algoritmos de optimización (SGD y Adam) y técnicas de regularización (con o sin *Dropout*).

Se ha utilizado la función de pérdida de Entropía Cruzada Binaria con penalización L2, utilizando un factor $\lambda = 10^{-4}$. La duración del entrenamiento se ha adaptado a la complejidad de cada arquitectura a medida que se han realizado las diferentes pruebas de entrenamiento: en el caso del MLP, se han realizado 100 épocas, mientras que la CNN se ha entrenado con 20 épocas, en vista del estancamiento de su aprendizaje en épocas posteriores. Por otro lado, el entrenamiento se ha realizado utilizando el Descenso de Gradiente Estocástico, con una tasa de aprendizaje $\eta = 10^{-2}$, y el método Adam, con $\eta = 10^{-4}$.

Cada modelo se ha entrenado de forma independiente bajo las cuatro configuraciones siguientes:

- **Experimento A:** Optimizador SGD sin *Dropout*.
- **Experimento B:** Optimizador SGD con *Dropout*.
- **Experimento C:** Optimizador Adam sin *Dropout*.
- **Experimento D:** Optimizador Adam con *Dropout*.

La implementación se ha desarrollado íntegramente en Python 3, contando con el modelo de inteligencia artificial Claude como asistente de programación para la estructuración y depuración del código, y apoyándose en un conjunto de librerías específico. Para el procesamiento y la gestión de los datos de las imágenes, se han utilizado librerías como OpenCV y NumPy. Para el modelado y entrenamiento de las redes, se ha utilizado PyTorch, actuando de motor principal del programa. Por último, se han utilizado librerías como Scikit-Learn y SciPy para la evaluación biométrica de los resultados ([*Comparativa_Redes_Firmas.ipynb*](#)).

6. Resultados prácticos

6.1 Curvas de aprendizaje: Evolución del coste frente al número de épocas

Para evaluar la dinámica de aprendizaje de cada uno de los modelos diseñados, se ha monitorizado el valor de la función de pérdida al final de cada época, de manera que se obtiene una comparativa visual de las curvas de entrenamiento (línea continua) y de validación (línea discontinua) para los ocho experimentos realizados (Figura 6.1).

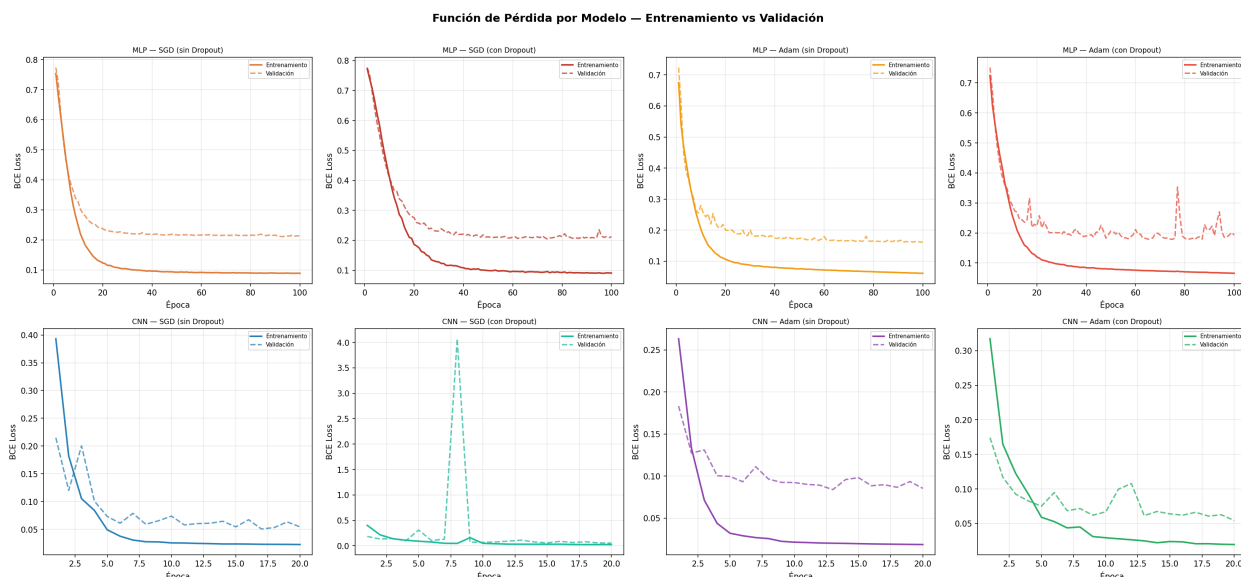


Figura 6.1: Gráficas de la función de pérdida de los experimentos realizados

6.1.1 Análisis del Perceptrón Multicapa

La fila superior de la Figura 6.1 muestra las curvas de aprendizaje correspondientes a las 100 épocas de entrenamiento realizadas sobre el MLP. Se observa un claro patrón de convergencia en los cuatro experimentos. Durante las 20 primeras épocas, el modelo aprende rápidamente, y las curvas del coste de aprendizaje y validación descienden de manera pronunciada. Sin embargo, a partir de la época 30, la pérdida de entrenamiento continúa descendiendo a menor velocidad hacia 0, mientras que la curva de validación se estanca en un valor cercano a 0,2. Esta separación entre ambas curvas indica un ligero sobreajuste (*overfitting*); al no poder

extraer características bidimensionales, la red comienza a memorizar la posición exacta de los píxeles de las imágenes de entrenamiento.

Cabe destacar el comportamiento inestable en el experimento de Adam con *Dropout*, en el que la agresividad de los picos en la curva de validación sugiere que se ha utilizado una tasa de aprendizaje adaptativa demasiado grande al combinarse con la desconexión aleatoria de neuronas.

6.1.2 Análisis de la Red Neuronal Convolutiva

La fila inferior de gráficas de la Figura 6.1, correspondiente a las 20 épocas realizadas sobre la CNN, muestra una gran superioridad en el uso de la Red Neuronal Convolutiva frente a los modelos anteriores. La convergencia es drásticamente más rápida, gracias a la extracción de características bidimensionales. A diferencia del MLP, la CNN logra estabilizar el error de validación en valores inferiores a 0,05, lo que se traducirá en una mayor seguridad biométrica. En el experimento con SGD y *Dropout*, se observa una anomalía en la octava época, en la que dispara el error de validación. Este fenómeno es habitual en el descenso de gradiente estocástico, y suele deberse a un lote de imágenes complejo utilizado, que genera un gradiente muy grande. No obstante, el modelo se recupera en la época inmediatamente posterior, retomando la convergencia.

El análisis de las funciones de pérdida demuestra empíricamente que la arquitectura de la CNN no solo entrena con una quinta parte de las épocas, sino que sufre un menor grado de memorización, y alcanza mínimos más profundos que el Perceptrón Multicapa.

6.2 Rendimiento global y matrices de confusión

Una vez finalizado el entrenamiento, se ha procedido a la evaluación definitiva de los ocho modelos frente al conjunto de prueba (*Test*), compuesto por 228 firmas (114 auténticas y 114 falsificadas). Para binarizar las probabilidades emitidas por la función sigmoide, se ha aplicado una función de Heaviside con un umbral de decisión fijado en $\tau = 0,5$. Es decir, si la probabilidad supera el 50 %, la firma se clasifica como auténtica, y viceversa.

Los resultados revelan una diferencia sustancial en el rendimiento entre las dos arquitecturas. El MLP obtuvo su mejor desempeño con Adam sin *Dropout*, alcanzando una exactitud (*Accuracy*) del 94,74 %. Su matriz de confusión registra 110 Verdaderos Positivos (*VP*) y 106 Verdaderos Negativos (*VN*). Sin embargo, cometió 4 Falsos Negativos (*FN*), es decir, firmas auténticas que fueron rechazadas por error; y, lo que es más crítico, 8 Falsos Positivos (*FP*) validando falsificaciones como si fueran auténticas. En su peor configuración (SGD con *Dropout*), el MLP llegó a aceptar 13 falsificaciones como auténticas.

Por el contrario, la CNN demostró una superioridad clara. Su “peor” modelo (Adam sin *Dropout*) logró una exactitud del 96,93 %, superando al MLP. El modelo ganador de toda la experimentación ha sido la CNN con SGD sin *Dropout*. Su exactitud alcanzó el 98,68 %. En su matriz de confusión, fue capaz de clasificar correctamente 112 firmas auténticas y 113 falsificaciones. Únicamente cometió 2 Falsos Negativos y tan solo 1 Falso Positivo con $\tau = 0,5$.

6.3 Análisis de Seguridad Biométrica

En el contexto de la verificación de firmas, se utilizan diferentes clasificadores y métricas especializadas en rendimiento biométrico [18]. En este ámbito, se asume que no todos los errores tienen el mismo impacto: el coste de un Falso Negativo (medido por la Tasa de Falso Rechazo o FRR) supone una molestia menor para el usuario, pero el coste de un Falso Positivo (medido por la Tasa de Falsa Aceptación o FAR) representa una vulnerabilidad crítica. Asimismo, para comparar la robustez general de distintos clasificadores independientes del umbral, se emplea la Tasa de Igual Error (EER), que representa el punto de equilibrio exacto entre ambas tasas. Las definiciones formales de dichas métricas son las siguientes:

$$FRR = \frac{FN}{FN + VP} \quad FAR = \frac{FP}{FP + VN} \quad EER = FAR(\tau^*) = FRR(\tau^*) \quad (6.1)$$

Analizando las tasas de error bajo el umbral $\tau = 0,5$, el modelo MLP presenta un FAR que oscila entre el 7,02 % y el 11,40 % según la configuración. En una aplicación real, permitir que cerca del 10 % de los intentos de fraude tengan éxito invalida el uso de esta arquitectura como sistema de seguridad.

En contraposición, la CNN con SGD sin *Dropout* reduce el riesgo biométrico a mínimos. Este modelo presenta un FRR del 1,75 % y un FAR del 0,88 %. El punto de equilibrio de este modelo (EER o *Equal Error Rate*), considerado como la métrica de referencia para la comparación de sistemas biométricos, se sitúa en un 0,88 %, indicando una mayor robustez cuanto más próximo a cero sea su valor. Aunque el modelo ya demuestra un desempeño excelente, es necesario priorizar la seguridad por encima de la comodidad del usuario. Por ello, se ha utilizado un valor $\tau = 0,8$, donde solo las firmas con una probabilidad superior al 80 % de ser auténticas son aceptadas. De este modo, los Falsos Negativos ascienden a 4, pero se logra evitar cualquier Falso Positivo, lo que se traduce en un FAR del 0 % y un FRR del 3,51 %.

6.4 Análisis de eficiencia y coste computacional

Más allá del rendimiento biométrico de los modelos, es relevante la evaluación del consumo de recursos y tiempo de ejecución del entrenamiento. En este contexto, la experimentación ha revelado una disparidad computacional notable entre ambas arquitecturas. Por un lado, los experimentos basados en el MLP mostraron una gran eficiencia temporal, obteniendo tiempos de ejecución inferiores a los 3 minutos para 100 épocas. Esta ligereza computacional se debe a la drástica reducción de dimensionalidad aplicada a las imágenes iniciales (compresión a 64×64 píxeles) y a la naturaleza de sus capas densas, que operan mediante multiplicaciones matriciales directas. Por otro lado, la CNN, al procesar imágenes de tamaño 320×320 píxeles, requirió alrededor de 45 minutos por experimento, incluso estando limitada a 20 épocas. Esta latencia se justifica por la elevada carga de procesamiento que exigen las convoluciones, donde múltiples filtros se deslizan iterativamente sobre la imagen de alta resolución.

A pesar de este notable incremento en el coste computacional durante la fase de desarrollo del modelo, la implementación de la CNN está plenamente justificada y sigue representando la opción idónea. El entrenamiento es un proceso que se ejecuta una única vez, pero la evaluación de una firma completa se realiza en tiempo real en fracciones de segundo, independientemente de la arquitectura. Por consiguiente, esta inversión de tiempo inicial se ve ampliamente compensada por la superioridad biométrica que ofrece la red convolucional.

6.5 Implementación en Aplicación móvil

Para demostrar la viabilidad y aplicabilidad del sistema en un escenario práctico, el estudio se ha complementado con el desarrollo de una aplicación móvil para el sistema operativo Android ([enlace de descarga](#)). Esta implementación se ha llevado a cabo utilizando el entorno Android Studio, con el soporte del modelo de inteligencia artificial Gemini como asistente de programación para estructurar y optimizar el código. El objetivo de esta herramienta es trasladar la capacidad predictiva del modelo ganador (CNN con SGD y sin *Dropout*) al usuario.

La aplicación (Figura 6.2) cuenta con una interfaz gráfica que permite al usuario capturar una fotografía de cualquier firma manuscrita o adjuntar una imagen desde la galería. Dado que la fiabilidad de la red depende de que los datos mantengan un mismo formato, el sistema incorpora un preprocesamiento interno que se ejecuta tras capturar una imagen. Primero se limpia el ruido de la imagen para mitigar imperfecciones. A continuación, se aplica un algoritmo que recorta la fotografía en el formato deseado, para posteriormente binarizar el archivo a blanco y negro puro. Por último, se redimensiona la imagen a 320×320 píxeles, y se transfiere directamente a la red CNN que ha mostrado el mejor rendimiento. El usuario

recibe por pantalla el porcentaje de probabilidad de que la firma sea auténtica.

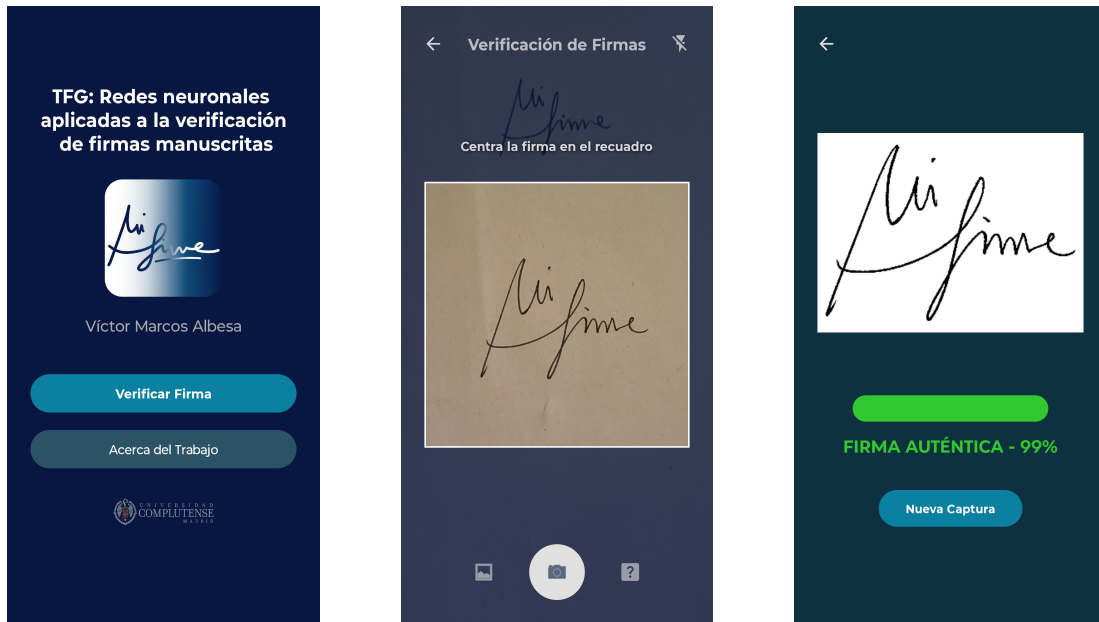


Figura 6.2: Aplicación móvil desarrollada. Interfaz de inicio de la aplicación (izquierda), captura de imagen (centro) y resultados (derecha).

7. Conclusiones

7.1 Conclusiones generales del proyecto

El desarrollo de este Trabajo de Fin de Grado ha permitido cumplir el objetivo principal propuesto: entender, estudiar, diseñar, entrenar y evaluar desde cero un sistema biométrico completo basado en redes neuronales capaz de distinguir firmas manuscritas auténticas de falsificaciones. A través de la experimentación, se ha logrado extraer conclusiones sólidas sobre el comportamiento de las distintas arquitecturas de redes neuronales estudiadas.

En primer lugar, se ha demostrado empíricamente la ineficacia de las redes neuronales clásicas para problemas complejos bidimensionales. El Perceptrón Multicapa, al requerir el aplanamiento de las imágenes y no poder extraer características bidimensionales, mostró una tendencia al sobreajuste y devolvió tasas de vulnerabilidad inaceptables, permitiendo que aproximadamente un 10 % de las falsificaciones se aceptasen como auténticas.

En contraposición, la implementación de la Red Neuronal Convolucional ha supuesto un gran salto de calidad. La capacidad de extracción de patrones espaciales de los filtros permitió converger más rápido y alcanzar una exactitud superior al 98 %. Específicamente, el optimizador de Descenso de Gradiente Estocástico (SGD) sin *Dropout* demostró ser el algoritmo más eficaz, obteniendo un FAR del 0 % al utilizar un umbral de decisión del 80 %. De este modo, se ha obtenido un clasificador robusto que erradica por completo la aceptación de falsificaciones, validando la hipótesis central de este proyecto.

7.2 Limitaciones de la implementación en un entorno realista y líneas de trabajo futuras

La mayor limitación estructural del enfoque que se ha tomado es que el modelo depende de una cantidad masiva de datos por cada usuario. Para que una red convolucional alcance métricas óptimas, es necesario entrenarla con un volumen considerable de muestras previas. En el contexto de este estudio, el sistema ha requerido cientos de firmas auténticas y, lo que resulta más restrictivo, cientos de falsificaciones para aprender a distinguir trazos fraudulentos de auténticos.

Si extrapolamos a un escenario real, como la apertura de una cuenta bancaria, la exigencia de

datos se vuelve inasumible. Una entidad no puede solicitar a un nuevo cliente que plasme su firma cien veces para registrarlo; y, lo que es más crítico aún, resulta materialmente imposible disponer de un conjunto de falsificaciones de ese cliente para entrenar a la red. Además, con la arquitectura de clasificación clásica, la incorporación de un nuevo usuario exige recopilar nuevas firmas y reentrenar completamente el modelo, lo cual es inviable en bases de datos masivas.

Por consiguiente, se concluye que, aunque la arquitectura CNN ha demostrado ser excepcionalmente precisa en un entorno controlado y estático, la clasificación binaria carece de escalabilidad. Como continuación natural de esta investigación, la principal línea de trabajo futuro se centra en la transición hacia la biometría dinámica o verificación *online*. El modelo desarrollado en este proyecto se basa exclusivamente en la biometría estática u *offline*, es decir, en el análisis de una imagen una vez que la firma ha sido plasmada y fotografiada.

Sin embargo, en el entorno profesional actual, como en las entidades bancarias, la captura de firmas se realiza mayoritariamente a través de pantallas táctiles avanzadas. Estos dispositivos no se limitan a registrar el trazo final, sino que capturan una serie de variables físicas y temporales en el momento en el que el usuario está firmando. Entre estas métricas dinámicas destacan la velocidad del trazo, la aceleración, las pausas al levantar el bolígrafo o la presión ejercida. Al incorporar estas variables dinámicas al modelo neuronal, se espera dar un salto importante en materia de seguridad. Un falsificador experto podría llegar a imitar la forma de una firma tras practicar, y engañar a una red convolucional; sin embargo, replicar simultáneamente la velocidad y los cambios de presión invisibles del autor original resulta prácticamente imposible.

Por ello, el desarrollo futuro de este proyecto pasaría por adaptar el sistema para pasar del análisis de fotografías al procesamiento de series temporales generadas por dispositivos electrónicos modernos. Integrar estas variables invisibles permitiría construir un sistema de verificación más seguro.

Bibliografía

- [1] Warren S McCulloch y Walter Pitts. «A logical calculus of the ideas immanent in nervous activity». En: *The bulletin of mathematical biophysics* 5.4 (1943), págs. 115-133.
- [2] Frank Rosenblatt. «The perceptron: A probabilistic model for information storage and organization in the brain». En: *Psychological review* 65.6 (1958), págs. 386-408.
- [3] Donald O Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.
- [4] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [5] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [6] George Cybenko. «Approximation by superpositions of a sigmoidal function». En: *Mathematics of control, signals and systems* 2.4 (1989), págs. 303-314.
- [7] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [8] Saad Albawi, Tareq Abed Mohammed y Saad Al-Zawi. «Understanding of a convolutional neural network». En: *2017 International Conference on Engineering and Technology (ICET)*. IEEE. 2017, págs. 1-6.
- [9] Entirety AI. *Convolutions in Deep Neural Networks*. <https://entiretydotai.github.io/blogs/deep%20neural%20network/convolutions/computer-vision/2020/07/06/Convolutions.html>. Accedido: 17 de marzo de 2024. 2020.
- [10] Vincent Dumoulin y Francesco Visin. «A guide to convolution arithmetic for deep learning». En: *arXiv preprint arXiv:1603.07285* (2016).
- [11] R Vijaya Kumar Reddy, B Srinivasa Rao y K Prudvi Raju. «Handwritten Hindi digits recognition using convolutional neural network with RMSprop optimization». En: *2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS)*. IEEE. 2018, págs. 45-51.
- [12] ProjectPro. *Introduction to Convolutional Neural Networks Algorithm and Architecture*. <https://www.projectpro.io/article/introduction-to-convolutional-neural-networks-algorithm-architecture/560>. Accedido: 23 de marzo de 2026. 2021.

- [13] Bernard Widrow y Marcian E Hoff. «Adaptive switching circuits». En: *IRE WESCON Convention Record*. Vol. 4. Institute of Radio Engineers. 1960, págs. 96-104.
- [14] David E Rumelhart, Geoffrey E Hinton y Ronald J Williams. «Learning representations by back-propagating errors». En: *Nature* 323.6088 (1986), págs. 533-536.
- [15] Diederik P Kingma y Jimmy Ba. «Adam: A method for stochastic optimization». En: *arXiv preprint arXiv:1412.6980* (2014).
- [16] Anders Krogh y John A Hertz. «A simple weight decay can improve generalization». En: *Advances in neural information processing systems*. Vol. 4. 1991.
- [17] Nitish Srivastava et al. «Dropout: a simple way to prevent neural networks from overfitting». En: *The journal of machine learning research* 15.1 (2014), págs. 1929-1958.
- [18] Anil K. Jain, Patrick Flynn y Arun A. Ross. *Handbook of biometrics*. Springer Science & Business Media, 2007.

Anexos

- Repositorio de GitHub:

<https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN.git>

- Programa funciones de activación:

https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/blob/main/otros_programas/funciones_de_activacion.ipynb

- Programa ejemplo de convolución:

https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/blob/main/otros_programas/ejemplo_de_convolucion.ipynb

- Programa comparativa de algoritmos de descenso:

https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/blob/main/otros_programas/comparativa_de_algoritmos_de_descenso.ipynb

- Plantilla firmas:

https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/blob/main/dataset/plantilla_firmas.pdf

- Programa extracción de imágenes del PDF:

https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/blob/main/dataset/extraccion_imagenes_dataset.ipynb

- Programa entrenamiento:

https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/blob/main/notebooks/Comparativa_Redes_Firmas.ipynb

- Aplicación móvil:

<https://github.com/victormarcosalbesa/TFG-Verificacion-Firmas-NN/releases/tag/v1.0-aplicacion-movil>